KHÓA THỰC HÀNH CURSOR • BUỔI 7 • 90 phút

Độ tin cậy của code do AI viết



Gỡ lỗi, refactor, đánh giá

Lấy lỗi của một cửa hàng online nhỏ do người khác viết làm ví dụ, học cách dựa vào căn cứ nào để tin code của AI.

0:00 • cả lớp

Trước khi bắt đầu

Trước khi vào bài, cả lớp hãy hoàn thành 3 việc sau.

- [ ] Mở `cursor-course/` và chạy `git pull`
- [ ] Chạy `node -v` trong terminal, kiểm tra kết quả hiện từ `v22` trở lên
- [ ] Chuyển chế độ phê duyệt sang **Auto-review** (Settings → Agents → Execution and Approvals)

Khi chạy `node -v` bị lỗi, hãy giơ tay. Nếu không có Node.js, test hôm nay sẽ không chạy được.

MỤC TIÊU HÔM NAY

Mục tiêu hôm nay

Chủ đề hôm nay: độ tin cậy của code do AI viết. Học cách làm cho code của AI trở nên đáng tin.
Căn cứ để tin không phải là lời của AI, mà là test, con mắt của người không phải người viết, yêu cầu và nhật ký.

1	**Yêu cầu AI điều tra lỗi của cửa hàng, tái hiện bằng test rồi mới sửa**	Chương 2–4
2	**Để một agent khác kiểm tra bản sửa của AI dựa trên yêu cầu**	Chương 5
3	**Xem AI sắp xếp lại code trong khi dùng test để giữ nguyên hành vi (GV demo)**	Chương 6
4	**Đối chiếu điều AI nói "đã làm" với điều AI thực sự đã làm, qua nhật ký**	Chương 7

> **Đạt được 1, 2 và 4 là hoàn thành.** Mục 3 sẽ được xem qua phần demo của GV.

HÔM NAY

Cách tiến hành hôm nay

Hôm nay, hãy giữ 3 điều sau khi làm.

01

Mỗi bước, hãy gọi đúng 1 Skill đã định sẵn

Mỗi Skill chỉ phụ trách 1 bước, và file được phép thay đổi cũng đã được quy định

02

Ở từng bước, hãy kiểm tra căn cứ cho lời của AI

AI nói “Đã sửa xong” thì vẫn chưa phải là căn cứ

03

Khi gặp khó khăn, hãy giơ tay

LỘ TRÌNH 90 PHÚT

Thời gian biểu

Giờ	Chương	Nội dung	Phụ trách
0:00	Chương 1 Mục tiêu hôm nay	Chủ đề và trình tự hôm nay (7 phút)	GV
0:07	Chương 2 Mở project	Sao chép, mở project và tự mình xem lỗi (8 phút)	Cả lớp
0:15	Chương 3 Điều tra	`/investigate` (10 phút)	Cả lớp
0:25	Chương 4 Tái hiện và sửa	`/reproduce` và `/fix` (18 phút)	Cả lớp
0:43	**Chương 5 Đánh giá**	/evaluate **và quyết định của người (20 phút)**	Cả lớp
1:03	Chương 6 Refactor	Xem `/refactor` qua phần demo của GV (12 phút)	GV
1:15	**Chương 7 Nhìn lại**	/review-trace **(10 phút)**	Cả lớp
1:25	Chương 8 Tổng kết	4 căn cứ, và các căn cứ đã chuẩn bị hôm nay (5 phút)	GV

Chương 5 và chương 7 không được cắt giảm. Chương 6 là phần demo của GV vì thời gian chờ AI dài.

CHƯƠNG 1 • Chủ đề hôm nay

Độ tin cậy của code do AI viết

Lauren Tan, tác giả của pstack (kỹ sư của Cursor), đã nói như sau trong một buổi chia sẻ.

01 **Agent có lúc tự tin khẳng định một nguyên nhân sai**

02 **Có lần agent khẳng định, nhưng chưa hề đọc phần code liên quan**

03 **Nếu agent không tự kiểm tra được, con người sẽ phải kiểm tra thay, và công việc bị dừng lại**

CHƯƠNG 1 • Chủ đề hôm nay

4 căn cứ để tin code của AI

Không căn cứ nào là lời của AI. Không chỉ dùng cho lỗi hôm nay, mà dùng được cho mọi code do AI viết.

Chương 4, 6

Test

Kiểm tra code có chạy như mong đợi không. AI cũng tự chạy được

+

Chương 5

Con mắt của người khác

Kiểm tra có sự chủ quan của người viết lẫn vào không

+

Chương 4, 5

Yêu cầu

Quyết định thế nào là đúng. Khi khác với test, yêu cầu là đúng

+

Chương 3, 7

Nhật ký

Kiểm tra AI có thực sự làm như đã nói không

> Hôm nay, lấy quá trình sửa lỗi #42 làm ví dụ, chúng ta sẽ chuẩn bị đủ 4 căn cứ này ở từng bước.

CHƯƠNG 1 • Ví dụ hôm nay (lỗi #42)

Mỗi bước gọi 1 Skill và kiểm tra căn cứ

Chương 3
Điều tra
`/investigate`
"Nguyên nhân ở đây"
→ Vị trí, mức độ căn cứ

▶

Chương 4
Tái hiện, sửa
`/reproduce`
`/fix`
"Đã sửa xong"
→ Test từ thất bại sang thành công

▶

Chương 5
Đánh giá
`/evaluate`
"Test đều thành công"
→ Người khác và yêu cầu

▶

Chương 6
Refactor
`/refactor`
"Hành vi không thay đổi"
→ Cùng các test đều thành công

▶

Chương 7
Nhìn lại
`/review-trace`
"Tôi đã làm ..."
→ Nhật ký không do AI viết

> Mọi Skill đều ghi mức độ căn cứ: `[said]` chỉ nói → `[line]` vị trí → `[traced]` lần theo → `[ran]` đã chạy

CHƯƠNG 2 • Cả lớp (4 phút)

Sao chép và mở project

1 Sao chép

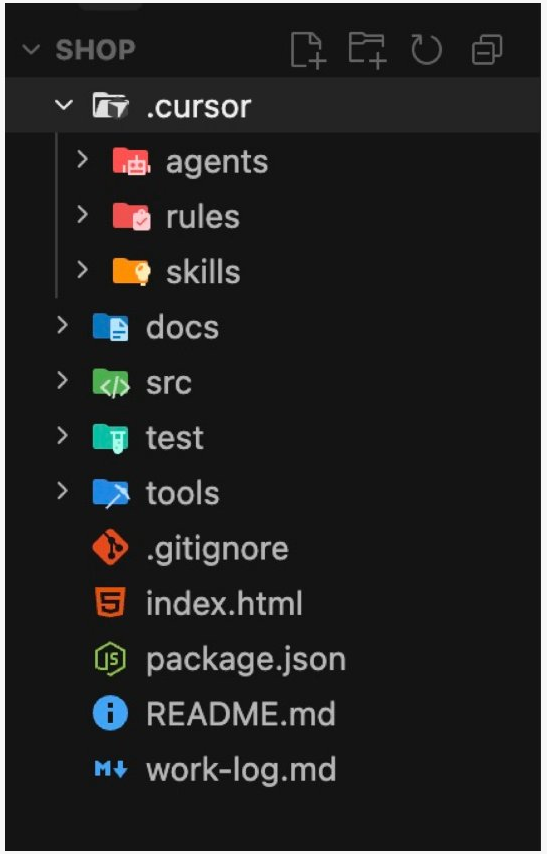
Trong chat: “Hãy sao chép `lessons/07/shop`, kể cả `.cursor`, ra Desktop với tên `shop`”

2 Mở thư mục

Chọn **File → Open Folder**, mở `shop` trên Desktop. Từ đây chỉ dùng cửa sổ này

3 Commit trạng thái ban đầu

Trong chat mới: “Hãy git init thư mục này, và commit tất cả file với message ‘bắt đầu’”



CHƯƠNG 2 • Cả lớp (4 phút)

Chạy test và xem lỗi

1 Chạy `node --test`

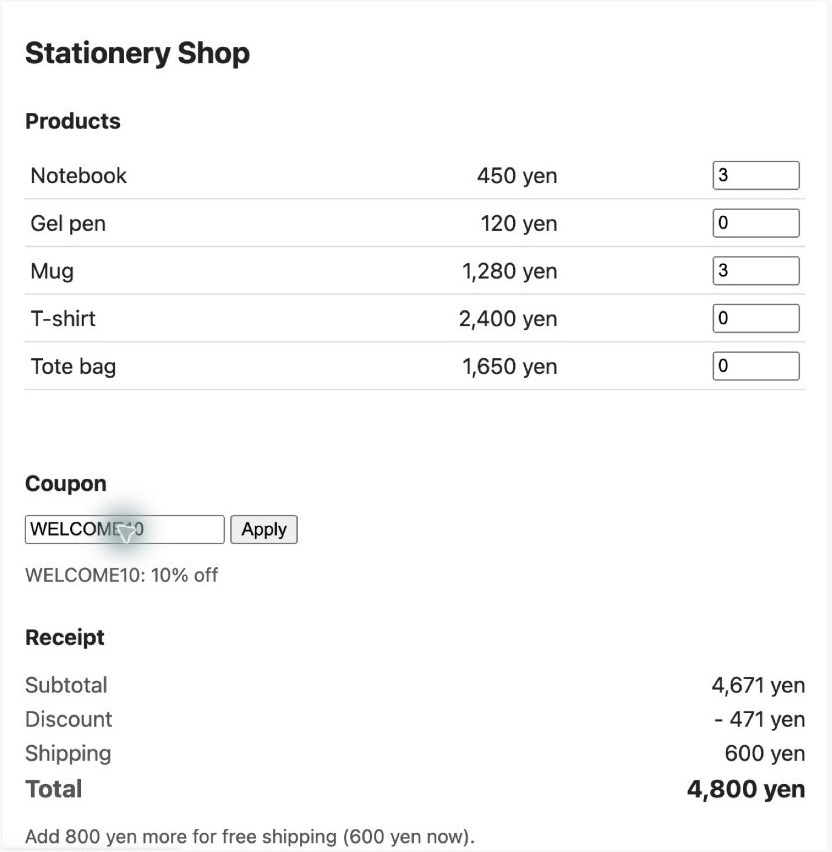
Hiện `# pass 13` và `# fail 0`. **Các test hiện có đều thành công**

2 Đọc `docs/bug-report.md`

Báo cáo cho biết: sau khi dùng WELCOME10, đổi số lượng thì subtotal bị sai

3 Mở `index.html` bằng trình duyệt tích hợp

Nhập WELCOME10 và nhấn Apply → đổi số lượng Mug thành 3. Subtotal phải là 3 × 1,280 + 3 × 450 = **5,190 yen**, nhưng lại nhỏ hơn



CHƯƠNG 2 • Khi gặp khó khăn

Cách xử lý khi gặp khó khăn

Vấn đề	Cách xử lý
Không tìm thấy node	Máy chưa cài Node.js. Hãy theo dõi màn hình của GV, và cài trong giờ nghỉ
Thư mục .cursor **không được sao chép**	Hãy yêu cầu lại: "Hãy sao chép cả thư mục `.cursor`"
Trình duyệt tích hợp hiện trang trắng	Mở file trực tiếp bằng trình duyệt của hệ điều hành thì việc tải sẽ bị chặn. Hãy mở bằng trình duyệt tích hợp của Cursor
Nhấn / mà không thấy investigate	Hãy kiểm tra thư mục đang mở có phải là `shop` không. Nếu vẫn mở `cursor-course/` thì sẽ không hiện

KHÓA THỰC HÀNH CURSOR • BUỔI 7

CHƯƠNG 3 · Câu hỏi của chương

Kiểm chứng câu “Nguyên nhân ở đây” của AI bằng gì?

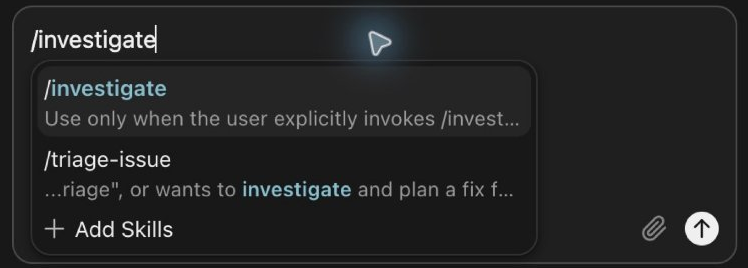
Căn cứ là vị trí được chỉ ra và mức độ căn cứ. Bạn cũng tự mở vị trí đó để kiểm tra.

CHƯƠNG 3 • Cả lớp (3 phút)

Yêu cầu điều tra bằng `/investigate`

Trong chat mới, chọn `investigate` từ `/` rồi gửi.

```
/investigate Hãy điều tra lỗi trong docs/bug-report.md.
```



Không viết dự đoán về nguyên nhân. Nếu viết dự đoán, AI sẽ chỉ tìm ở chỗ đó.

CHƯƠNG 3 • Màn hình

Câu trả lời của `/investigate`

	Kết quả điều tra
Nguyên nhân	WELCOME10 ghi đơn giá đã giảm vào dòng giỏ. Lần tính sau cộng từ giá đó, nên mỗi lần đổi số lượng subtotal lại thấp hơn.
Vị trí	`applyCoupon` trong `src/coupons.js`, dòng `item.price = newPrice`. `calculateTotal` chỉ đọc giá đó.
Bằng chứng	[traced] Đổi số lượng gọi `setQuantity` rồi `calculateTotal`, và hàm này gọi lại `applyCoupon`. [ran] Sau WELCOME10, giá cốc trong giỏ là 1.152, danh mục vẫn 1.280. Đổi thành 3 cốc: subtotal 4.671, giá niêm yết 5.190.

> Hãy xem mỗi nguyên nhân có ghi vị trí (file và hàm) và mức độ căn cứ không. Những điều cần kiểm tra có ở trang sau.

CHƯƠNG 3 • Cả lớp (7 phút)

Kiểm tra câu trả lời

Chỗ cần xem	Điều cần kiểm tra
Phần diễn đạt lại	Có khớp với báo cáo không. Có ghi “SAVE500 không bị” và “tải lại trang thì đúng” không
Quy định liên quan	Có trích dẫn dòng trong `docs/spec.md` không
Các nguyên nhân có thể	Có ghi vị trí (file và hàm) không. Có kèm mức độ căn cứ (`[said]`〜`[ran]`) không
Những điều chưa rõ	Có ghi không. Nếu không có, hãy ghi lại điều đó

> **Hãy chọn 1 nguyên nhân, và tự mở vị trí được chỉ ra.** Hãy kiểm tra xem code có đúng như đã ghi không.

KHÓA THỰC HÀNH CURSOR • BUỔI 7

CHƯƠNG 4 · Câu hỏi của chương

Kiểm chứng câu “Đã sửa xong” của AI bằng gì?

Căn cứ là test thất bại trước khi sửa và thành công sau khi sửa. Giá trị mong đợi được tính từ yêu cầu.

CHƯƠNG 4 • Giải thích

Tái hiện rồi mới sửa

Trước khi sửa, ta viết trước một test thất bại vì lỗi đó. **Việc chính test đó chuyển sang thành công là bằng chứng đã sửa xong.**

`/reproduce`

Tính giá trị mong đợi từ yêu cầu

3 × 1,280 + 3 × 450 = 5,190 yen. Không dùng giá trị có được khi chạy code hiện tại

▶

Test thất bại

expected 5190 actual 4671

Kiểm tra test thất bại đúng vì lý do trong báo cáo, rồi chỉ commit test

▶

`/fix`

Sửa `src/`

Không thay đổi file test

▶

Chính test đó thành công

`# fail 0`

Output chuyển từ thất bại sang thành công là bằng chứng đã sửa. Commit code đã sửa

> Lịch sử còn lại theo thứ tự “test thất bại” → “code đã sửa”, nên người xem sau cũng kiểm tra được là đã sửa xong.

CHƯƠNG 4 • Cả lớp (5 phút)

Yêu cầu viết test thất bại bằng `/reproduce`

Trong cùng chat với chương 3, hãy gửi tiếp.

```
/reproduce Hãy tái hiện lỗi vừa điều tra bằng test.
```

Giá trị mong đợi phải **tính từ yêu cầu, không lấy từ việc chạy code.** Nếu ghi giá trị của code hiện tại, ta sẽ coi hành vi sai là đúng.

```
# Subtest: a quantity change after WELCOME10
not ok 7 - a quantity change after WELCOME10
  error: |-
    Expected values to be strictly equal:

    4671 !== 5190

  expected: 5190
  actual: 4671
# tests 14
# pass 13
# fail 1
```

CHƯƠNG 4 • Kiểm tra câu trả lời

Kiểm tra test thất bại

Chỗ cần xem	Điều cần kiểm tra
Giá trị mong đợi	Có tính từ `docs/spec.md` không (ví dụ: 3 × 1,280 + 3 × 450 = 5,190)
Output khi thất bại	Test mới có thất bại, và có hiện `expected` và `actual` không
Lý do thất bại	Có thất bại vì cùng lý do với báo cáo không (subtotal nhỏ hơn 5,190)
File đã thay đổi	Có phải chỉ là file trong `test/` không

> Sau khi kiểm tra, hãy chỉ commit file test với message `test: reproduce bug #42`. **Lưu test thất bại trước khi sửa.**

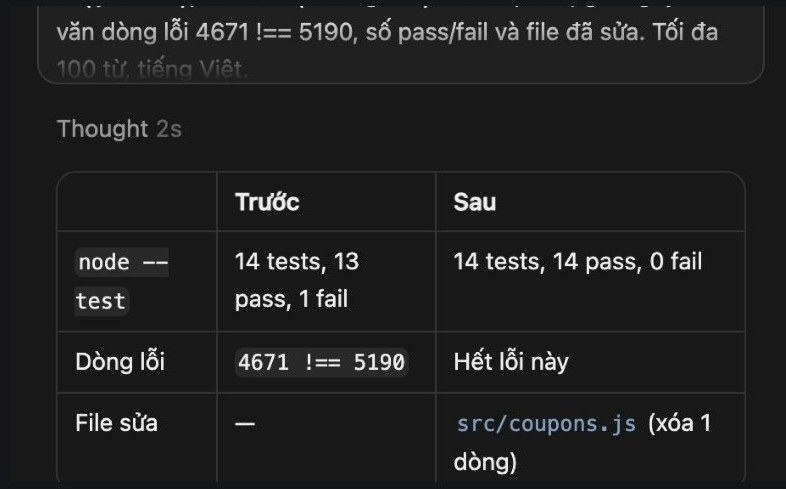
CHƯƠNG 4 • Cả lớp (7 phút)

Yêu cầu sửa bằng `/fix`

Trong cùng chat, hãy gửi tiếp.

```
/fix Hãy sửa lỗi vừa tái hiện.
```

`/fix` chỉ thay đổi `src/`. **Không thay đổi test.** Nếu cho rằng test sai, AI sẽ dừng lại và chỉ ra dòng trong yêu cầu.


văn dòng lỗi 4671 !== 5190, số pass/fail và file đã sửa. Tối đa 100 từ, tiếng Việt.

Thought 2s

	Trước	Sau
`node --test`	14 tests, 13 pass, 1 fail	14 tests, 14 pass, 0 fail
Dòng lỗi	`4671 !== 5190`	Hết lỗi này
File sửa	—	`src/coupons.js` (xóa 1 dòng)

CHƯƠNG 4 • Kiểm tra câu trả lời

Kiểm tra việc sửa nguyên nhân gốc

Chỗ cần xem	Điều cần kiểm tra
Cách xác nhận nguyên nhân	Có chạy để kiểm tra từng nguyên nhân một không. Có quyết định bằng phỏng đoán không
Chỗ đã sửa	Có sửa `applyCoupon` trong `src/coupons.js` không. Có chặn ở code màn hình (`src/app.js`) không
Output thất bại → thành công	Test thất bại trước khi sửa giờ có thành công không. Tất cả test có thành công không
File đã thay đổi	`git diff --stat` có chỉ hiện file trong `src/` không

CHƯƠNG 4 • Giải thích

Cơ chế của lỗi #42

`applyCoupon` đã ghi đè lên chính giá của sản phẩm trong giỏ hàng. Nếu theo dõi giá của Mug, ta sẽ thấy như sau.

Ban đầu

Mug 1,280 yen

Mug 2, Notebook 3, subtotal là 3,910 yen

▶

Apply WELCOME10

Mug 1,152 yen

Giảm giá 391 yen hiện đúng. Nhưng giá trong giỏ hàng bị ghi đè thành giá đã giảm 10%

▶

Đổi Mug thành 3

Subtotal 4,671 yen

Vì tính bằng giá đã bị ghi đè, nên nhỏ hơn 5,190 yen

▶

Sau đó

Mug 1,036 yen

Mỗi lần tính lại, giá lại bị ghi đè giảm thêm 10%

> Tải lại trang thì hết lỗi, vì giỏ hàng được tạo lại với giá ban đầu.

CHƯƠNG 4 • Giải thích

Cách sửa che triệu chứng, và cách sửa nguyên nhân gốc

Che triệu chứng

Chặn ở phía màn hình (`app.js`): "chỉ giảm giá 1 lần".

Test có thể thành công. Nhưng nếu gọi `applyCoupon` 2 lần từ chỗ khác, lỗi sẽ lại xảy ra.

Sửa nguyên nhân gốc

Sửa để `applyCoupon` không ghi đè lên giá của sản phẩm.

Vì giá bị ghi đè, nên mỗi lần tính lại thì giảm giá lại bị cộng dồn.

> Triệu chứng xuất hiện ở màn hình, nhưng **chỗ làm sai đầu tiên là `applyCoupon`.**

CHƯƠNG 4 • Cả lớp (5 phút)

Đọc diff và commit

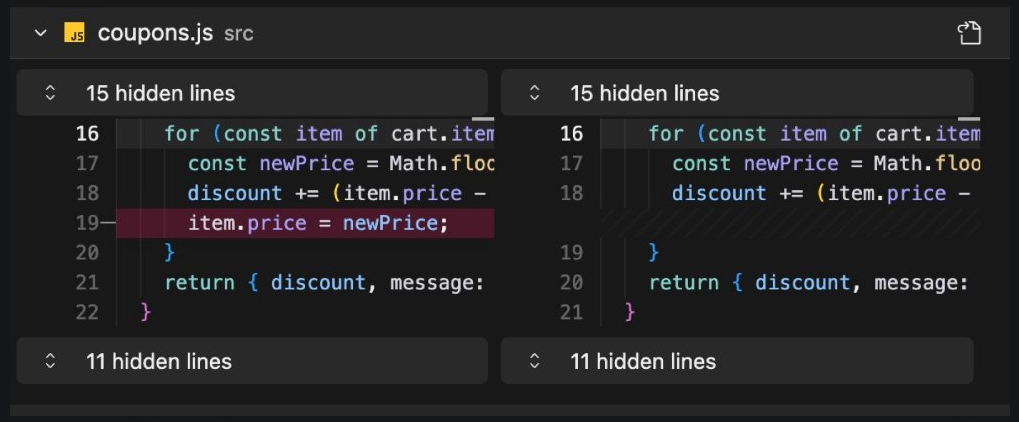
- [ ] Hãy đọc thay đổi trong `src/` ở panel Source Control, và kiểm tra cả việc file test không bị thay đổi
- [ ] Hãy tự chạy `node --test` trong terminal và xác nhận `# fail 0`
- [ ] Hãy commit với message `fix: bug #42 ...`
- [ ] Trong lịch sử, commit test thất bại và commit sửa lỗi nằm theo đúng thứ tự này

Nếu `/fix` đã thay đổi file test, hãy gửi: "Hãy trả file test về như cũ, và chỉ sửa trong `src/`".

CHƯƠNG 4 • Màn hình

Diff đã sửa, và thứ tự trong lịch sử

Thay đổi ở `src/coupons.js`



Lịch sử

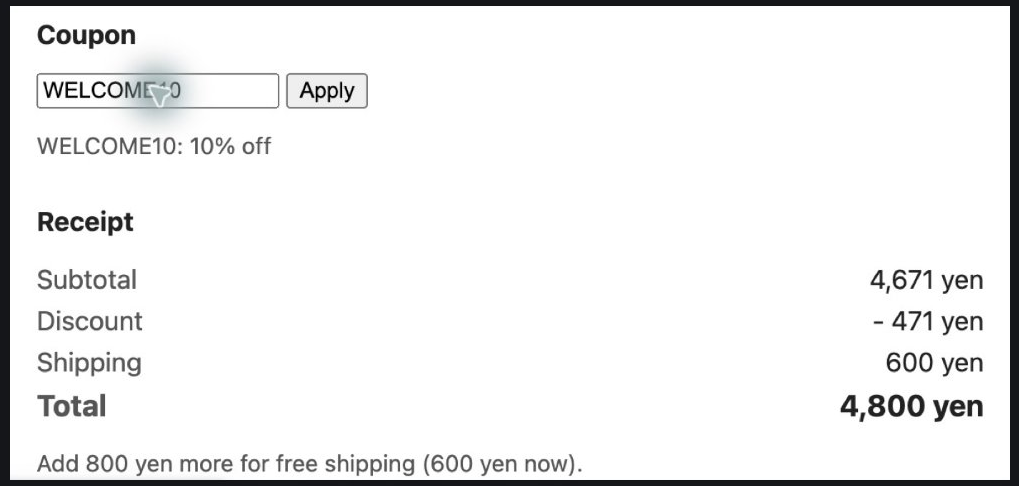


> Chỉ có file trong `src/` thay đổi. Trong lịch sử, commit test thất bại đứng trước, commit đã sửa đứng sau.

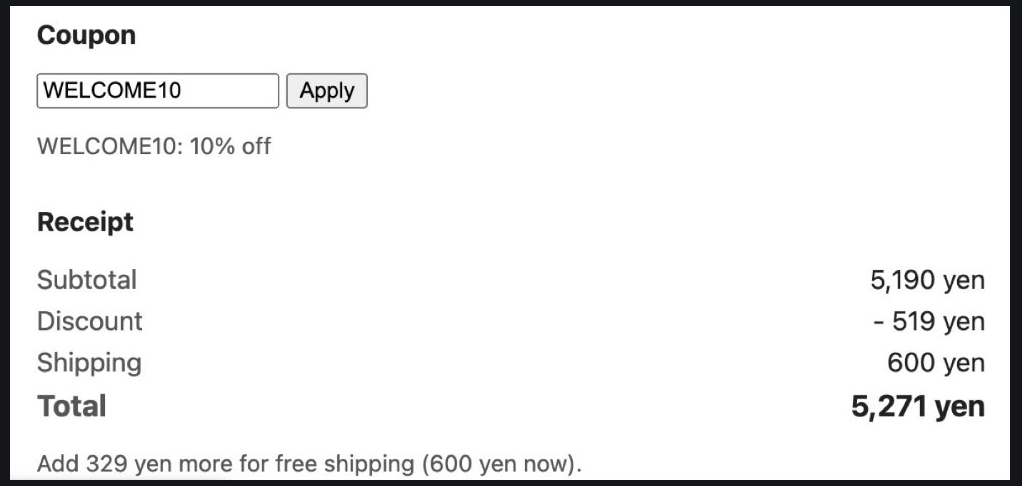
CHƯƠNG 4 • Màn hình

Trước khi sửa, và sau khi sửa

Trước khi sửa



Sau khi sửa



> So sánh bằng cùng một thao tác: nhấn Apply với WELCOME10, rồi đổi số lượng Mug thành 3. Sau khi sửa, dù tính lại bao nhiêu lần, subtotal vẫn không thay đổi.

KHÓA THỰC HÀNH CURSOR • BUỔI 7

CHƯƠNG 5 · Câu hỏi của chương

Chỉ “Test đều thành công” là đã đủ chưa?

Căn cứ là con mắt của người không phải người sửa, và yêu cầu. Nếu test khác với yêu cầu, hãy nghi ngờ test.

CHƯƠNG 5 • Giải thích

Việc mà `evaluator` làm

Mở `.cursor/agents/evaluator.md` cho cả lớp xem. Không đưa lời giải thích của người đã sửa.

#	Việc làm	Điều đang kiểm tra
1	Đọc yêu cầu, mục đích thay đổi, và commit đã sửa	Thế nào là đúng
2	Chạy tất cả test	Test hiện tại có thành công không
3	**Cố ý làm hỏng code đã sửa**, xem test có thất bại không. File bị làm hỏng được trả lại bằng git	Test có phát hiện lại lỗi này không
4	**Viết test mới từ yêu cầu** (giá trị đúng ngưỡng, tính lại 2 lần, giỏ hàng trống…)	Chỗ đã sửa và xung quanh có đúng yêu cầu không
5	Khi thất bại, tìm xem code hay test khác với yêu cầu. Không sửa bên nào	Cần sửa cái gì
6	Đưa ra kết luận (`VERIFIED` · `NOT VERIFIED` · `INCONCLUSIVE`)	`INCONCLUSIVE` không phải là đạt

CHƯƠNG 5 • Màn hình

Mở `evaluator.md`

Read-only Background

```
1  ---
2  name: evaluator
3  description: Use only when the evaluate skill or the user explicitly
   asks the evaluator to check a committed fix. Checks the fix against
   docs/spec.md, tests whether the tests can catch a broken version, and
   writes new tests from the spec in test/eval/. Never keeps a change to
   src/ or to existing tests.
4  model: inherit
5  readonly: false
6  ---
```

> `readonly: false` là để viết test vào `test/eval/`, cố ý làm hỏng `src/` rồi trả lại như cũ.

CHƯƠNG 5 • Giải thích

Cố ý làm hỏng để kiểm tra test

Đây là bước 3 của evaluator. Tạm thời làm hỏng code đã sửa, rồi xem test có phát hiện ra không.

Làm hỏng

Đưa chỗ đã sửa về như trước khi sửa

Tạm thời viết lại `src/coupons.js`

▶

Chạy

`node --test`

Chạy toàn bộ test

▶

Xem

Test có thất bại không

Nếu thất bại, test đang phát hiện được lỗi này. Nếu vẫn thành công, test không bảo vệ được gì

▶

Trả lại

Trả lại bằng `git checkout`

File đã làm hỏng thì nhất định phải trả lại như cũ

> Nếu sau khi đánh giá mà `src/` vẫn còn thay đổi, nghĩa là evaluator đã quên trả lại.

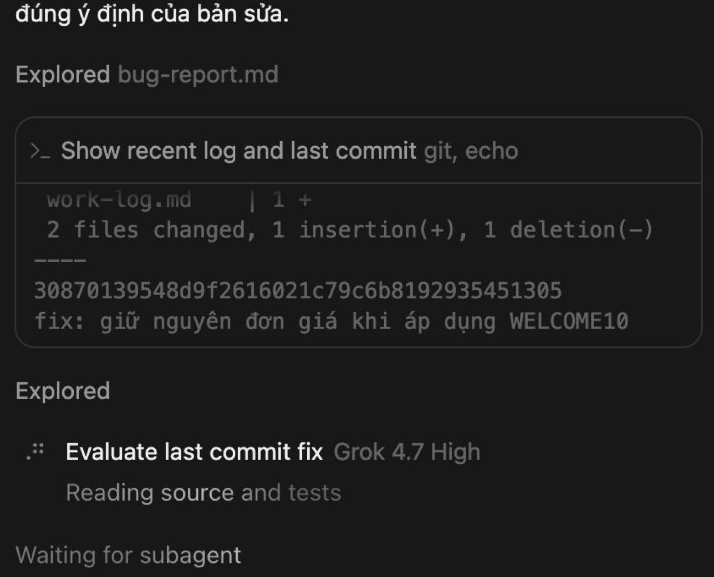
CHƯƠNG 5 • Cả lớp (3 phút)

Gửi `/evaluate`

Hãy gửi trong **chat mới**, để cuộc hội thoại lúc sửa không ảnh hưởng đến việc đánh giá.

```
/evaluate Hãy đánh giá bản sửa trong commit cuối cùng.
```

Nếu AI báo "có thay đổi chưa commit", hãy commit xong ở chương 4 rồi gửi lại.



CHƯƠNG 5 • Màn hình

Kết quả đánh giá

Verdict: NOT VERIFIED. `node --test`: 27 test, 25 đạt, 2 trượt. WELCOME10 đúng 3.000: giảm 0, tổng 3.600 (kỳ vọng 300, 3.300). SAVE500 đúng 5.000: giảm 0, ship 0, tổng 5.000 (kỳ vọng 500, 600, 5.100).

Vấn đề	Bằng chứng	Đề xuất
WELCOME10 từ chối đúng 3.000 yên	Spec: «3,000 yen or more». Code: `<= 3000`. Test cũ `WELCOME10 needs a purchase over 3,000 yen` kỳ vọng giảm 0 (4 sổ + 10 bút = 3.000).	Đổi `<=` thành `<`. Sửa test cũ thành giảm 300.
SAVE500 từ chối đúng 5.000 yên	Spec: «5,000 yen or more». Code: `<= 5000`. Không trừ 500 nên ship 0.	Đổi `<=` thành `<`. Giữ `shipping.js`.

> Hãy xem kết luận và Act on (những gì cần sửa), rồi chép vào bảng ở trang sau.

CHƯƠNG 5 • Cả lớp (7 phút)

Ghi lại kết quả đánh giá

Hãy ghi kết quả đánh giá vào bảng sau (ghi vào sổ tay của mình cũng được).

Mục	Kết quả
Kết luận	VERIFIED ／ NOT VERIFIED ／ INCONCLUSIVE
Test thất bại khi cố ý làm hỏng	
Test mới bị thất bại	
Act on (cần sửa)	

> Trong nhiều trường hợp, kết luận sẽ là **NOT VERIFIED**.

CHƯƠNG 5 • 2 điều tìm ra khi đánh giá

Test cũng có thể sai

1 Một lỗi khác

WELCOME10 không dùng được khi **đúng 3,000 yen**.

Yêu cầu là "3,000 yen or more (từ 3,000 yen trở lên)". Code lại là "lớn hơn 3,000 yen". SAVE500 cũng vậy.

2 Test sai

Test có sẵn "WELCOME10 needs a purchase **over** 3,000 yen" đang kiểm tra rằng khi đúng 3,000 yen thì giảm giá là 0.

Đây là test coi hành vi sai là đúng.

> **Thứ quyết định đúng sai là yêu cầu.** Vì vậy evaluator lấy yêu cầu làm tiêu chuẩn, không phải test.

CHƯƠNG 5 • Giải thích

So sánh ở giá trị biên

So sánh mức giảm của WELCOME10 giữa yêu cầu và code. **Chỉ khác nhau khi đúng 3,000 yen.**

2,990 yen

Yêu cầu: 0 yen
Code: 0 yen

Ít hơn 3,000 yen, nên cả hai đều không giảm giá

Đúng 3,000 yen

Yêu cầu: 300 yen
Code: 0 yen

Yêu cầu ghi "trở lên". Code ghi "từ 3,000 trở xuống thì không giảm"

3,910 yen (giỏ hàng ban đầu)

Yêu cầu: 391 yen
Code: 391 yen

Ví dụ trong buổi học là số tiền này, nên chỉ nhìn màn hình thì không nhận ra

> Test có sẵn đã kiểm tra rằng đúng 3,000 yen thì giảm 0. **Test cũng mắc cùng lỗi với code.**

CHƯƠNG 5 • Cả lớp (10 phút)

Người quyết định, rồi sửa

1 Quyết định bên nào đúng (1 phút)

Xem dòng “A subtotal of 3,000 yen or more” trong bảng Coupons của `docs/spec.md`. **Người quyết định: yêu cầu đúng, code và test cũ sai**

2 Cho phép sửa test cũ (3 phút)

`/fix` không được thay đổi test. Test chỉ được sửa khi người đã quyết định (trang sau)

3 Yêu cầu sửa bằng `/fix` (4 phút)

Hãy kiểm tra test thất bại giờ đã thành công, và `git diff --stat` chỉ hiện `src/coupons.js`

4 Commit riêng (2 phút)

Tách commit sửa test và commit sửa code

CHƯƠNG 5 • ② Cho phép sửa test cũ

Người quyết định, rồi yêu cầu sửa test

Trong cùng chat, hãy gửi nội dung sau.

```
Test “WELCOME10 needs a purchase over 3,000 yen” trong test/coupons.test.js
không khớp với “3,000 yen or more” trong docs/spec.md.
Hãy chỉ sửa test này cho đúng với yêu cầu:
đúng 3,000 yen thì giảm giá là 300 yen,
2,990 yen (Notebook 3・Gel pen 3・Mug 1) thì là 0.
Hãy chạy test và cho thấy bây giờ test thất bại.
```

> AI có khi **sửa test** để test thành công. Hãy xem dòng trong yêu cầu, người quyết định “test này sai” rồi mới yêu cầu sửa.

CHƯƠNG 5 • ③ Yêu cầu sửa bằng `/fix`

Yêu cầu sửa điều kiện số tiền tối thiểu

Trong cùng chat, hãy gửi tiếp.

```
/fix Hãy sửa điều kiện số tiền tối thiểu của WELCOME10 và SAVE500
thành “từ … trở lên”, đúng như docs/spec.md.
```

> SAVE500 cũng có cùng cách viết sai. **Hãy tìm xem còn lỗi cùng dạng ở chỗ khác không.**

CHƯƠNG 5 • Khi gặp khó khăn

Cách xử lý khi gặp khó khăn

Vấn đề	Cách xử lý
evaluator **không được gọi**	Hãy gửi: “Hãy gọi subagent evaluator theo đúng bước 3 của evaluate”
Sau khi đánh giá, src/ **vẫn còn thay đổi**	evaluator đã quên trả lại file. Hãy hủy thay đổi trong `src/`, giữ lại file mới trong `test/eval/`
evaluator đã sửa test cũ	Điều này trái với quy định. Hãy hủy thay đổi đó, rồi người quyết định cho phép và yêu cầu sửa
Test trong test/eval/ **vẫn thất bại**	Hãy đọc tên test và dòng trong yêu cầu, rồi quyết định bên nào đúng theo cùng cách

KHÓA THỰC HÀNH CURSOR • BUỔI 7

CHƯƠNG 6 · Câu hỏi của chương

Kiểm chứng câu “Hành vi không thay đổi” của AI bằng gì?

Căn cứ là toàn bộ các test giống như trước khi refactor đều thành công.

CHƯƠNG 6 • GV demo (6 phút)

Yêu cầu refactor bằng `/refactor`

Hãy gửi trong **chat mới**.

```
/refactor Số tiền miễn phí vận chuyển (5000), phí vận chuyển (600) và quy định coupon
đang được viết ở nhiều file. Hãy gom về 1 chỗ.
```

Hiện tại `5000` và `600` có ở cả `shipping.js` và `app.js`. Chỉ sửa 1 chỗ thì thông báo trên màn hình sẽ khác với phí vận chuyển thực tế.

CHƯƠNG 6 • Màn hình

Câu trả lời của `/refactor`

Quy tắc	Trước	Sau
Ngưỡng miễn phí vận chuyển (`5000`)	3	1
Phí vận chuyển (`600`)	2	1
Nhánh mã coupon	2	1 bảng

```
# tests 28
# pass 28
# fail 0
```

> Hãy xem số chỗ viết cùng một quy định có giảm từ trước sang sau không, và kiểm tra cả số test và kết quả có giống trước khi refactor không.

CHƯƠNG 6 • Giải thích

Các bước tiến hành của `/refactor`

Trong Skill có ghi rằng phải tiến hành theo thứ tự sau.

1

Cố định hành vi hiện tại bằng test

Nếu thiếu test, chỉ được thêm ở bước này

2

Đếm số chỗ viết cùng một quy định

`5000` và `600` có ở cả `shipping.js` và `app.js`

3

Gom lại từng chút một

Mỗi bước đều chạy test. Trước khi thêm, xóa những gì có thể xóa

4

So sánh trước và sau

Số chỗ có giảm không, số test thành công có giữ nguyên không. Nếu không dễ đọc hơn thì trả lại như cũ

CHƯƠNG 6 • GV demo (6 phút)

So sánh trước và sau

Chỗ cần xem	Điều cần kiểm tra
Số chỗ trước và sau	Số chỗ viết cùng một quy định có giảm không
Kết quả test	Số test giống như trước khi refactor, và tất cả đều thành công không
File test	`git diff --stat test/` có thay đổi nào ở test có sẵn không
Hành vi mới	Có tính năng nào không được yêu cầu mà bị thêm vào không

> Mở lại `index.html` bằng trình duyệt tích hợp, xem hóa đơn và thông báo phí vận chuyển giống như trước, rồi commit với `refactor: ...`.

KHÓA THỰC HÀNH CURSOR • BUỔI 7

CHƯƠNG 7 · Câu hỏi của chương

Kiểm chứng báo cáo của AI có đúng không bằng gì?

Căn cứ là nhật ký không do AI viết (lịch sử git và lịch sử hội thoại của Cursor).

CHƯƠNG 7 • Giải thích

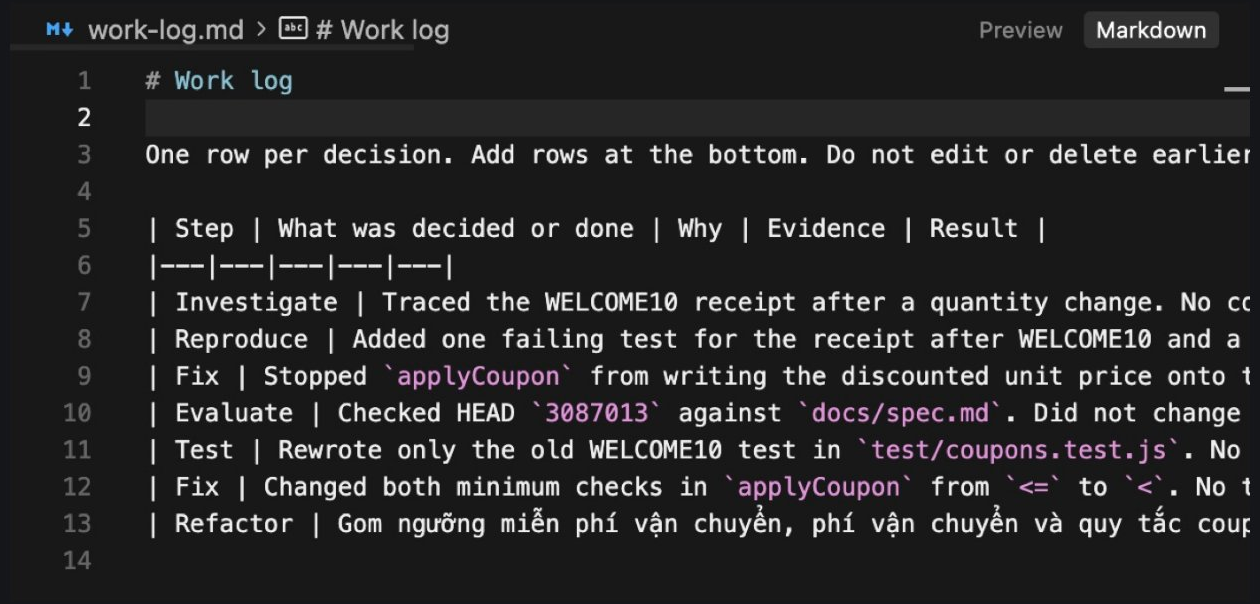
Những gì AI nói, và những gì AI đã thực sự làm

Nhật ký	Ai viết	Biết được gì
work-log.md **và câu trả lời của AI**	Chính AI	AI nói rằng mình đã làm gì
Lịch sử git (git log --stat**)**	git	Commit nào đã thay đổi file nào
Lịch sử hội thoại của Cursor	Cursor	AI đã thực sự đọc file nào, ghi file nào, chạy lệnh nào

> Đối chiếu 2 loại này sẽ thấy được “nói mà không làm” và “làm mà không nói”. **Đây là khả năng quan sát của hôm nay.**

CHƯƠNG 7 • Màn hình

Mở `work-log.md`



> Mỗi khi xong một bước, AI ghi thêm 1 dòng. **Vì đây là nhật ký AI tự viết, nên chỉ dựa vào nó thì không biết AI có thật sự làm hay không.**

CHƯƠNG 7 • Giải thích

Đối chiếu 2 loại nhật ký

Nhật ký AI viết

`work-log.md` và phản hồi

AI nói rằng mình đã làm gì

+

Nhật ký Cursor để lại

Lịch sử hội thoại

File AI thật sự đã đọc, file đã viết, lệnh đã chạy

▶

`/review-trace`

Xếp theo từng bước

Dùng `tools/trace.mjs` xếp các lần gọi công cụ, rồi so cả với lịch sử git

▶

Kết quả

Đưa chỗ không khớp vào Attention

Nếu không đọc được lịch sử hội thoại, kết quả sẽ là `INCONCLUSIVE`

> Nhờ vậy, ta tìm ra được “nói mà không làm” và “làm mà không nói”.

CHƯƠNG 7 • Cả lớp (2 phút)

Đối chiếu bằng `/review-trace`

Hãy gửi trong **chat mới**.

```
/review-trace Hãy nhìn lại công việc hôm nay.
```

Khi không đọc được lịch sử hội thoại, kết quả sẽ hiện `INCONCLUSIVE`. Khi đó, hãy xem kết quả kiểm tra chỉ bằng lịch sử git và `work-log.md`.

CHƯƠNG 7 • Màn hình

Câu trả lời của `/review-trace`

Fix #42	Sửa `src/coupons.js`; `node --test` trước và sau; git `3087013` xóa 1 dòng	match
Evaluate	Phá `src/` rồi `git checkout` 2 lần; thêm `test/eval`	match
Test mốc	Sửa `test/coupons.test.js`; git `106e507` không có `src/`	match
Fix mốc	Hai sửa `src/coupons.js`; git `c9087b5` chỉ coupons và work-log	match
Refactor	Chỉ sửa `src/`; `node --test` 3 lần; git `d50b174` không có test	match

Attention

- Stdout không có trong transcript, nên số pass/fail không được xác minh độc lập.
- Bỏ trim và chữ hoa thì không test nào fail; work-log không ghi. Hai commit đầu không có lệnh commit trong transcript.

> Ở từng bước, hãy xem báo cáo của AI có khớp (match) với git và lịch sử hội thoại không. **Người đối chiếu cũng là AI, nên dù match vẫn phải tự kiểm tra nội dung báo cáo.**

CHƯƠNG 7 • Cả lớp (6 phút)

Kiểm tra bằng bảng trong câu trả lời

Điều cần kiểm tra	Chỗ cần xem
Ở bước AI nói "test đã thành công", AI có thực sự chạy node --test **không**	Dấu `RAN TESTS`
/fix **và** /refactor **có thay đổi test có sẵn không**	Dấu `CHANGED A TEST`
/investigate **có thay đổi file không**	Dấu `CHANGED SRC`
Có việc nào không có trong work-log.md **không**	Danh sách Attention
Số lượng test có khớp với báo cáo của AI không	Kết quả tự chạy `node --test`

> **Hãy chọn 1 mục trong danh sách Attention và tự kiểm tra.** Ví dụ, mở commit được chỉ ra trong panel Source Control.

CHƯƠNG 8 • Tổng kết

Các căn cứ đã chuẩn bị hôm nay

Bước	AI đã nói	Căn cứ đã kiểm chứng
Điều tra	“Nguyên nhân ở đây”	Vị trí và mức độ căn cứ. Tự mình đã mở ra kiểm tra
Tái hiện và sửa	“Đã sửa xong”	Test đã thất bại trước khi sửa, và thành công sau khi sửa
Đánh giá	“Test đều thành công”	evaluator không phải người sửa, và yêu cầu. Đã tìm ra một test sai
Refactor	“Hành vi không thay đổi”	Đúng các test như trước khi refactor đều đã thành công
Nhìn lại	“Tôi đã làm …”	Nhật ký không do AI viết (lịch sử git và lịch sử hội thoại của Cursor)

> Việc con người đã làm có 3 điều: **kiểm tra AI có hiểu sai báo cáo không, xem yêu cầu để quyết định bên nào đúng, và kiểm tra báo cáo của AI bằng nhật ký**.

CHƯƠNG 8 • Bài tập về nhà

Hãy chọn 1 lần AI không làm đúng quy định, và thêm 1 dòng vào Skill đó để ngăn việc này

Chỗ cần thêm là file `SKILL.md` của Skill đó.

Lần sau làm cùng công việc, hãy thử xem dòng đó có tác dụng không.

CHƯƠNG 8 • GÓC MỞ RỘNG

Nguồn gốc của các Skill hôm nay (pstack)

Skill hôm nay	Phần làm cùng việc trong pstack
/investigate	Playbook Investigation, `/how`
/reproduce • /fix	Playbook Bug fix, `/tdd`
/evaluate • evaluator	`/interrogate` (review bằng cách để model khác thử làm hỏng)
/refactor	Playbook Refactoring
/review-trace • work-log.md	`/show-me-your-work` (đối chiếu nhật ký quyết định với lịch sử hội thoại)

> pstack do nhân viên Cursor làm ra: mỗi khi thấy agent mắc lỗi, họ thêm một Skill để ngăn lỗi đó. **Bài tập thêm 1 dòng vào Skill là trải nghiệm nhỏ cách làm đó.**

Khóa thực hành Cursor

Tổng kết hôm nay

Độ tin cậy của code do AI viết không quyết định bằng lời của AI, mà bằng căn cứ: test, con mắt của người không phải người viết, yêu cầu và nhật ký.

Khi AI nói "Đã sửa xong", đừng tin cho đến khi thấy bằng chứng. Bằng chứng là output test chuyển từ thất bại sang thành công.

Test cũng có thể sai. **Thứ quyết định đúng sai là yêu cầu.**

Những gì AI nói và những gì AI đã làm, hãy ghi lại riêng và đối chiếu.